TEC | Tecnológico de Costa Rica

Documentación técnica del Proyecto 1: Monopoly Distribuido con Estructuras Lineales

Jimenez Corella Johan Daniel

Mendoza Baranovicht Adriel De Jesus

Rodriguez Madrigal Jefferson Ariel

Sandi Arrieta Jose David

Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computación

CE 1103 : Algoritmos y Estructuras de Datos 1

Ellioth Ramirez Trejos

9 de Octubre de 2026

Tabla de contenido

Documentación de las estructuras de datos utilizadas 3 Lista circular doblemente enlazada 3 Descripción general 3 Uso en el proyecto 3 Cola circular 5 Descripción general 5 Uso en el proyecto 5 Lista simplemente enlazada 6 Descripción general 6 Uso en el proyecto 6 Conexión cliente-servidor en Monopoly distribuido 7 Propósito y arquitectura 7 Componentes principales 8 Canales y parámetros de red 8 Establecimiento de la conexión 9 Arranque y descubrimiento 9 Streams y bienvenida 9 Respuestas, eventos y concurrencia 10 Protocolo de mensajes 10 Uso, integración y comportamiento observado 12 Flujo de una partida 12 Integración con el cajero 12 Diagramas UML 13 Errores y límites de la implementación actual 14 Referencias 16 Apéndice 17 Inventario del código fuente consultado 17

Documentación técnica del Proyecto 1: Monopoly Distribuido con Estructuras Lineales

Este documento técnico reúne la documentación del Proyecto 1, Monopoly Distribuido con Estructuras Lineales, del curso Algoritmos y Estructuras de Datos 1. Primero se describen las estructuras de datos lineales implementadas para el tablero, los turnos y las propiedades de los jugadores. Después se explica la comunicación entre cliente y servidor, con su protocolo de mensajes y los diagramas UML de clases y de secuencia. El manual breve de ejecución se entrega en un documento aparte y el módulo de hardware se entrega junto con el repositorio del proyecto.

Documentación de las estructuras de datos utilizadas

En el proyecto se implementaron tres estructuras de datos lineales propias: una lista circular doblemente enlazada para el tablero, una cola circular para los turnos y una lista simplemente enlazada para las propiedades de cada jugador. En ninguna se usaron las clases List, LinkedList, Stack o Queue de Java, como lo pide el enunciado del proyecto (Instituto Tecnológico de Costa Rica [TEC], 2026).

Lista circular doblemente enlazada

Descripción general

La lista circular doblemente enlazada es una estructura lineal formada por nodos. Cada nodo guarda un dato y dos referencias: una al nodo anterior y otra al siguiente. Como es circular, el último nodo apunta a la cabeza y la cabeza apunta al último, por lo que la lista no tiene un final con referencia nula y se puede recorrer en los dos sentidos.

Uso en el proyecto

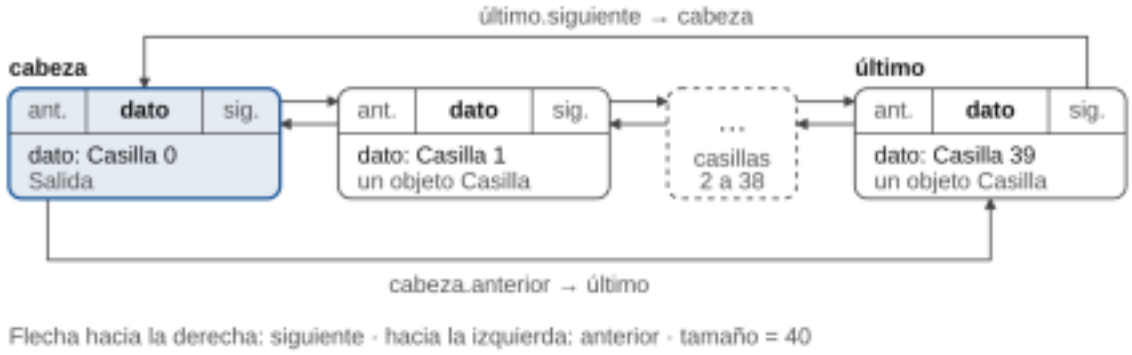
En el proyecto esta lista se utilizó para representar el tablero. La clase ListaCircularDoble<Casilla> se guarda en Tablero en el atributo casillas y contiene 40 casillas; la

casilla con índice 0 es la Salida. Cada nodo es un objeto Nodo<T> con tres atributos: dato, que

guarda una Casilla (puede ser una Propiedad, una CasillaEvento o una CasillaEspecial), siguiente, que apunta al nodo que sigue, y anterior, que apunta al nodo previo. La Figura 1 muestra la estructura del tablero.

Figura 1

Lista circular doblemente enlazada del tablero



Nota. Se muestran el primer nodo, el segundo, el último y, de forma abreviada, los intermedios. Elaboración propia.

Basado en la Figura 1, la cabeza guarda la casilla Salida en su campo dato. Su referencia siguiente apunta a la casilla 1 y su referencia anterior apunta al último nodo, la casilla 39. Así se cierra el círculo, porque el siguiente del último nodo vuelve a apuntar a la cabeza. Se escogió esta estructura porque el jugador avanza casilla por casilla y, al pasar de la última, sigue en la Salida sin necesitar un caso especial. Además, las cartas que hacen retroceder al jugador usan la referencia anterior. Los métodos más importantes para el juego son agregarAlFinal, que agrega una casilla al final, y avanzar y retroceder, que mueven al jugador siguiendo los enlaces siguiente o anterior.

Cola circular

Descripción general

La cola es una estructura lineal en la que el primer elemento que entra es el primero que sale (FIFO). En la cola circular se usa un arreglo de tamaño fijo y los índices avanzan con el

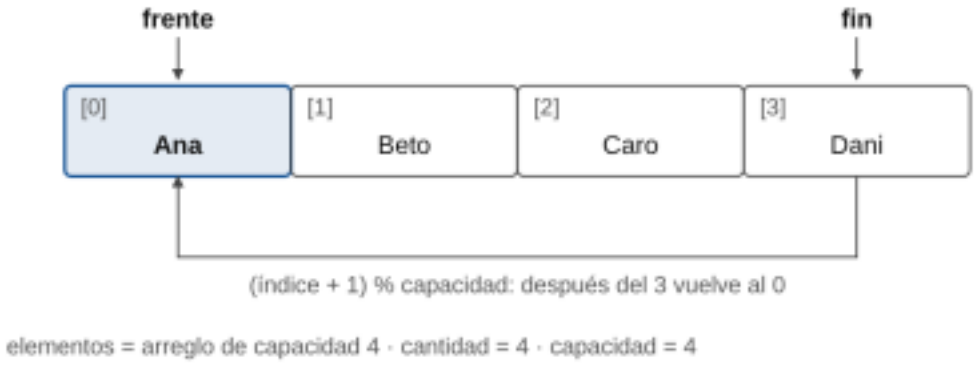
operador módulo, de manera que al llegar a la última posición se vuelve a la primera y no hay que mover los elementos.

Uso en el proyecto

La cola circular se utilizó para manejar los turnos. La clase ColaCircular<Jugador> se guarda en Juego en el atributo turnos y su capacidad se define al crearla; en una partida de cuatro jugadores su capacidad es cuatro. Esta cola no usa nodos enlazados, sino un arreglo Object[] elementos y tres atributos: frente, que es el índice del jugador que juega a continuación; fin, que es el índice del último jugador agregado; y cantidad, que es el número de jugadores que hay en la cola. Los índices avanzan con la fórmula (indice + 1) % capacidad. La Figura 2 muestra cómo se organiza el arreglo.

Figura 2

Cola circular de turnos implementada con un arreglo



Nota. Los nombres de los jugadores son un ejemplo ilustrativo. Los jugadores ocupan posiciones del arreglo y no hay punteros entre ellos; el orden de atención (FIFO) va de frente a fin. Elaboración propia.

Basado en la Figura 2, el jugador que está en frente es el que tiene el turno. Juego lo consulta con verFrente() y, cuando el turno termina, lo saca con desencolar(). Si el jugador sigue activo, se vuelve a agregar al final con encolar(), así el orden se mantiene y el siguiente jugador queda al frente. Si fin está en la última posición, el índice siguiente vuelve a 0, como muestra la

flecha de abajo. Un jugador eliminado o en bancarrota se saca de la cola y no se vuelve a agregar. Esta estructura sirve para los turnos porque encolar, desencolar y verFrente no necesitan mover los demás elementos.

Lista simplemente enlazada

Descripción general

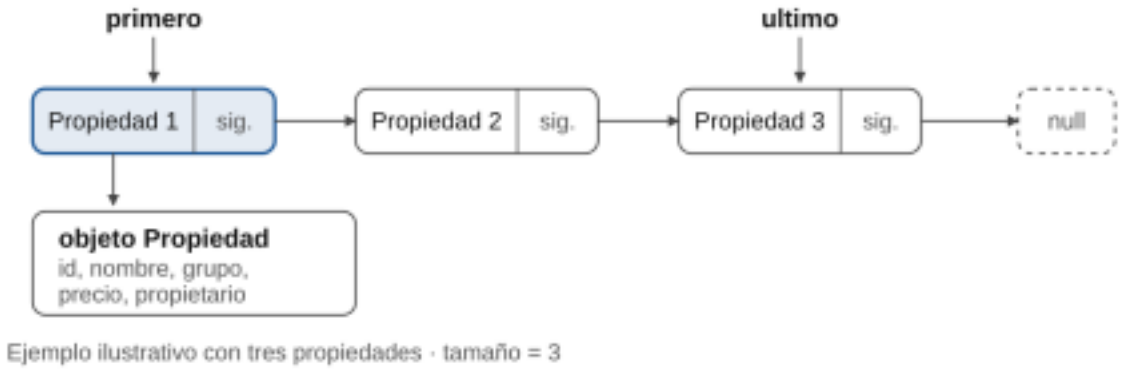
La lista simplemente enlazada es una estructura formada por nodos, donde cada nodo guarda un dato y una sola referencia al nodo siguiente. El último nodo apunta a null. Solo se puede recorrer en un sentido, desde el primer nodo hasta el último.

Uso en el proyecto

Esta lista se utilizó para guardar las propiedades que compra cada jugador. La clase ListaSimple<Propiedad> se guarda en Jugador en el atributo propiedades y mantiene las propiedades en el orden en que se adquieren. Cada nodo es un objeto NodoSimple<T> con dos atributos: dato, que referencia a una Propiedad, y siguiente, que referencia al nodo que sigue o es null si es el último. La lista también guarda las referencias primero y ultimo y el contador tamaño. La Figura 3 muestra un ejemplo con tres propiedades.

Figura 3

Lista simplemente enlazada de las propiedades de un jugador



Nota. Los nombres de las propiedades son ilustrativos; el campo dato de cada nodo referencia a un objeto Propiedad. Elaboración propia.

Basado en la Figura 3, primero apunta a la propiedad más antigua y ultimo a la más reciente, y el último nodo apunta a null. Como la lista guarda ultimo, agregar una propiedad al final no obliga a recorrerla. Antes de agregar, el método agregarPropiedad de Jugador usa contiene para revisar que la propiedad no esté repetida. Los métodos calcularValorPropiedades, contarCasas y contarHoteles recorren la lista desde primero. Si un jugador cae en bancarrota, entregarTodasLasPropiedades pasa las propiedades a un arreglo y vacía la lista. Se usó una lista simple porque todas estas operaciones recorren las propiedades en un solo sentido.

Conexión cliente-servidor en Monopoly distribuido

Propósito y arquitectura

Esta sección describe la implementación de red existente, tomando Interfaz.java como punto de entrada. La revisión es estática: se consultó el código fuente sin ejecutar ni modificar el proyecto. El servidor conserva el estado oficial; los clientes envían solicitudes y muestran los datos que reciben. El análisis se basa en *Monopoly distribuido: Código fuente de la conexión cliente-servidor* (s. f.), disponible en el proyecto local.

Componentes principales

La Tabla 1 resume los componentes de la conexión y sus responsabilidades.

Tabla 1

Componentes de la conexión cliente-servidor

Componente	Responsabilidad
Interfaz Inicio	de Swing, arranque opcional del servidor, registro de usuarios, creación/unión a partidas y registro RFID.
Cliente	Descubre el servidor, abre TCP, envía mensajes y separa respuestas de eventos mediante un hilo lector.
ManejadorCliente	Atiende un socket de computadora; identifica la solicitud y delega en Servidor.
Servidor / modelo.Juego	Valida solicitudes, coordina el estado de la partida, aplica reglas y publica los resultados.

Descubrimiento Localiza el servidor por UDP sin pedir su dirección IP al usuario.

cliente.VentanaTablero Recibe eventos mediante callbacks y envía acciones mediante Consumer<Mensaje>.

Nota. Elaboración propia a partir de las clases del proyecto; véase el Apéndice.

Canales y parámetros de red

La Tabla 2 resume los canales de comunicación y los parámetros configurados.

Tabla 2

Canales y parámetros de red

Canal Puerto Formato / finalidad

Descubrimiento UDP 5001 Texto UTF-8: buscar el servidor en la red local. Juego TCP 5100 Objetos Java serializados de protocolo.Mensaje. Cajero TCP 5050 Líneas de texto separadas por punto y coma. Búsqueda 5000 ms Límite configurado para buscar el servidor. Respuesta de gestión 10 segundos Espera de Cliente.recibirRespuesta(). *Nota.* Elaboración propia a partir de las clases del proyecto; véase el Apéndice.

La configuración contempla dos computadoras y cuatro jugadores: dos por computadora. El organizador puede ejecutar servidor y cliente en la misma máquina. Los valores provienen de config.Configuracion y Cliente.

Establecimiento de la conexión

Arranque y descubrimiento

1. Interfaz.main() intenta crear un Cliente y conectarlo. Si la conexión falla, deja cliente en null. El botón de servidor inicia Servidor.iniciar() en otro hilo y después intenta conectar el cliente local.

2. Servidor abre un ServerSocket para el juego, anuncia el servicio mediante Descubrimiento y abre el canal del cajero en un hilo separado. Acepta dos conexiones de computadora y entrega cada una a un ManejadorCliente mediante un pool de dos hilos.

3. Cliente.conectar() llama a buscarServidor(5000). La búsqueda envía

DISCOVER_GAME_REQUEST a las direcciones de broadcast de las interfaces activas y a 127.0.0.1, siempre al puerto UDP 5001.

4. El servidor responde GAME_HERE: seguido del puerto TCP del juego, normalmente 5100. El cliente obtiene la IP del remitente del datagrama y usa esa IP junto con el puerto anunciado para abrir el Socket TCP.

Streams y bienvenida

5. Cliente y ManejadorCliente crean primero ObjectOutputStream y ejecutan flush(); después construyen ObjectInputStream. Este orden permite intercambiar los encabezados de serialización antes de que las entradas bloqueen esperando datos.

6. El manejador envía Bienvenida(computadora), con índice 0 o 1 según el orden de conexión. El cliente guarda ese índice y arranca un hilo lector daemon. El manejador registra su stream de salida en Servidor para permitir mensajes espontáneos.

Respuestas, eventos y concurrencia

El hilo lector del cliente es el único que lee los objetos entrantes después de la bienvenida. Los objetos Respuesta se agregan a una LinkedBlockingQueue; registrar(), crearPartida() y entrarPartida() esperan en esa cola hasta diez segundos. Los demás Mensaje se entregan al Consumer registrado con setOyente().

Cliente guarda el último EstadoJuego para entregarlo cuando cambia el oyente. Durante el escaneo, Interfaz escucha ResultadoTarjeta; al abrir el tablero enlaza cliente::enviarAccion y ventana::recibirMensaje. Las actualizaciones de componentes se trasladan al hilo de Swing mediante invokeLater().

Las acciones de la partida se protegen con synchronized(gameLock). La escritura del servidor usa synchronized(out), writeObject(), reset() y flush(); reset() evita reutilizar referencias de objetos previamente enviados. El envío del cliente también es synchronized y vacía el stream

con flush().

Protocolo de mensajes

protocolo.Mensaje es una interfaz sealed que extiende Serializable. Sus variantes son records. Cliente y servidor deben disponer de las mismas clases y versiones compatibles; el canal del juego no utiliza JSON ni líneas de texto.

La Tabla 3 resume las solicitudes que envía el cliente.

Tabla 3

Solicitudes del cliente al servidor

Solicitud: cliente → servidor	**Contenido / resultado**
Registrarse	nombre → Respuesta: registro correcto, duplicado o error.
CrearPartida	jugador1, jugador2 → Respuesta; el mensaje devuelve el código de cuatro dígitos.
UnirsePartida	codigo, jugador1, jugador2 → Respuesta; al completarse se inicia Juego.
RegistrarTarjeta	jugadorLocal (0 o 1) → ResultadoTarjeta, inmediatamente ante un error o después de la lectura RFID.
TirarDadosPrueba	Sin campos. Solicita una tirada generada en el servidor para probar sin el cajero.
ComprarPropiedad / ComprarCasa / ComprarHotel	Propiedad actual o idPropiedad para casa/hotel. El servidor valida y puede dejar un pago RFID pendiente.
TerminarTurno	Sin campos. El servidor valida y pasa el turno.
PagarMultaCarcel / UsarCartaCarcel	Sin campos. Solicitan salir de la cárcel según las reglas.
ConsultarTransacciones	jugador, tipo y recientesPrimero; los filtros pueden ser null.
ExportarTransacciones	Sin campos → HistorialTransacciones con texto y nombre de archivo.

Nota. Elaboración propia a partir de las clases del proyecto; véase el Apéndice.

La Tabla 4 resume los mensajes que recibe el cliente.

Tabla 4

Mensajes del servidor al cliente

Servidor → cliente	**Datos principales**
Bienvenida	Número de computadora asignado.
Respuesta	tipo y mensaje para operaciones de registro y partida. ResultadoTarjeta jugadorLocal, registrada, uid y mensaje.
Dados	indiceJugador, dado1 y dado2.
Aviso	Texto de información o error.
EstadoJuego	Turno, jugadores, saldos, posiciones, computadoras, propiedades, cárcel, disponibilidad del banco y fin de partida.
HistorialTransacciones	texto, nombreArchivo y bandera exportacion.

Nota. Elaboración propia a partir de las clases del proyecto; véase el Apéndice.

EstadoPropiedad es un record Serializable incluido dentro de EstadoJuego; no es un mensaje independiente. Las solicitudes de juego normalmente producen Aviso y/o un nuevo estado, no una Respuesta de la cola de gestión. ManejadorCliente ignora los tipos destinados exclusivamente a salir del servidor si un cliente los envía.

Uso, integración y comportamiento observado

Flujo de una partida

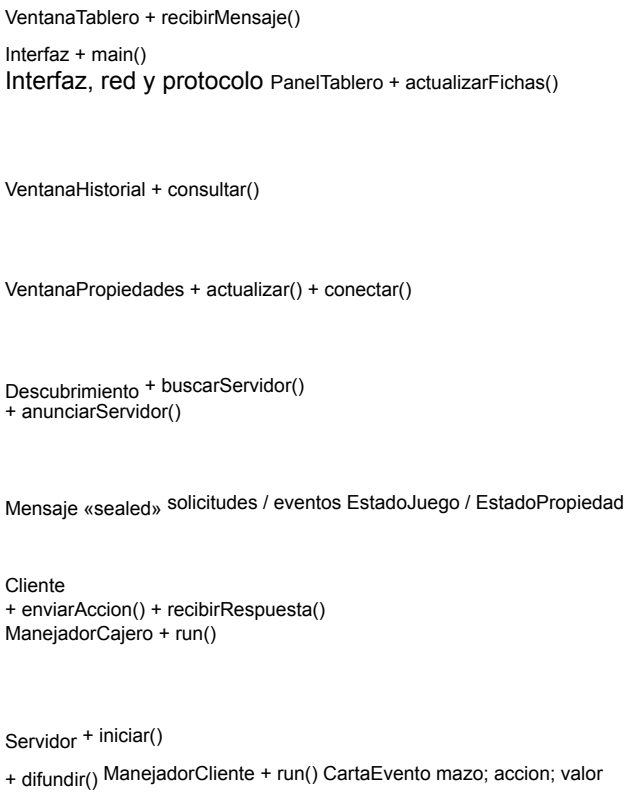
Iniciar el servidor desde Interfaz o desde Servidor.main(); abrir Interfaz en las dos computadoras; registrar los usuarios; crear una partida con dos nombres; compartir el código de cuatro dígitos y unirse desde la otra computadora con sus dos jugadores. Al unirse se crea el juego y se difunde su estado. La interfaz guía el registro RFID antes de mostrar el tablero.

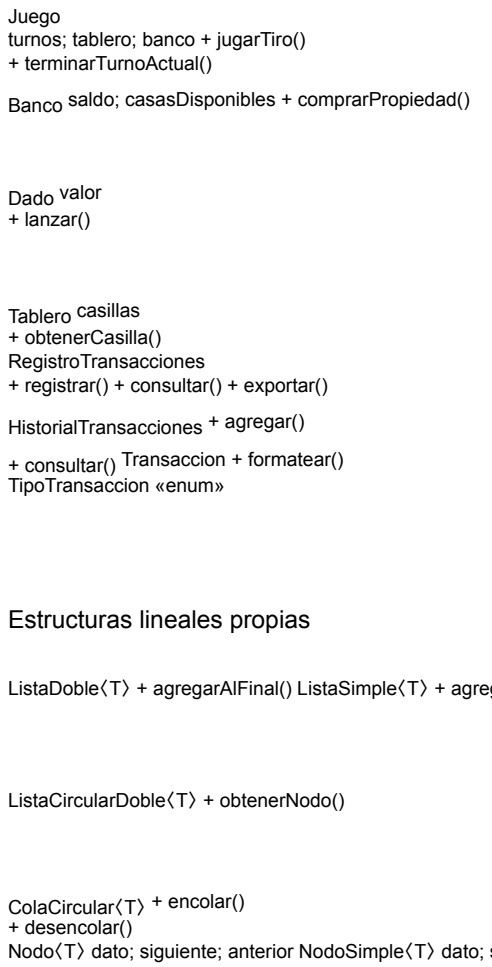
Integración con el cajero

ManejadorCajero escucha una conexión TCP aparte en 5050. Reconoce HOLA;CAJERO, DADOS;d1;d2 y TARJETA;UID. Responde BIENVENIDO, OK_DADOS, OK_TARJETA o ERROR;MOTIVO según el procesamiento. Servidor recibe los datos y publica

Diagramas UML del proyecto completo

Figura 4. Diagrama UML de clases — Monopoly distribuido Incluye interfaz, red, modelo, reglas, transacciones y estructuras lineales.





Nota. Asociaciones: flechas continuas; dependencias: discontinuas; herencia: triángulo vacío. Fuente: código Java del proyecto.

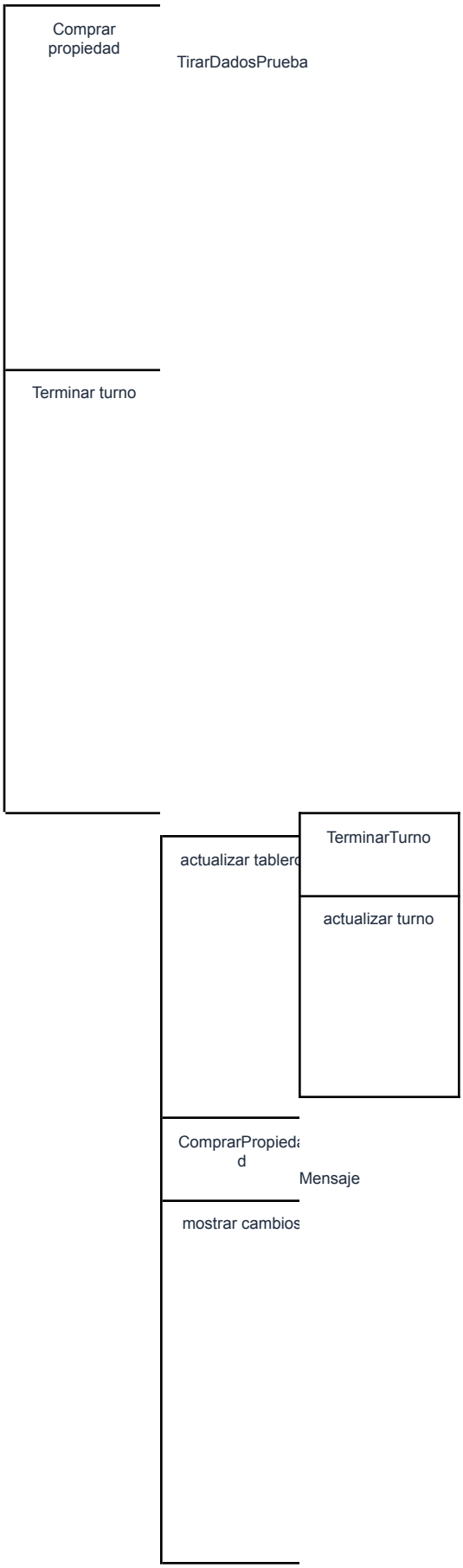
Diagramas UML

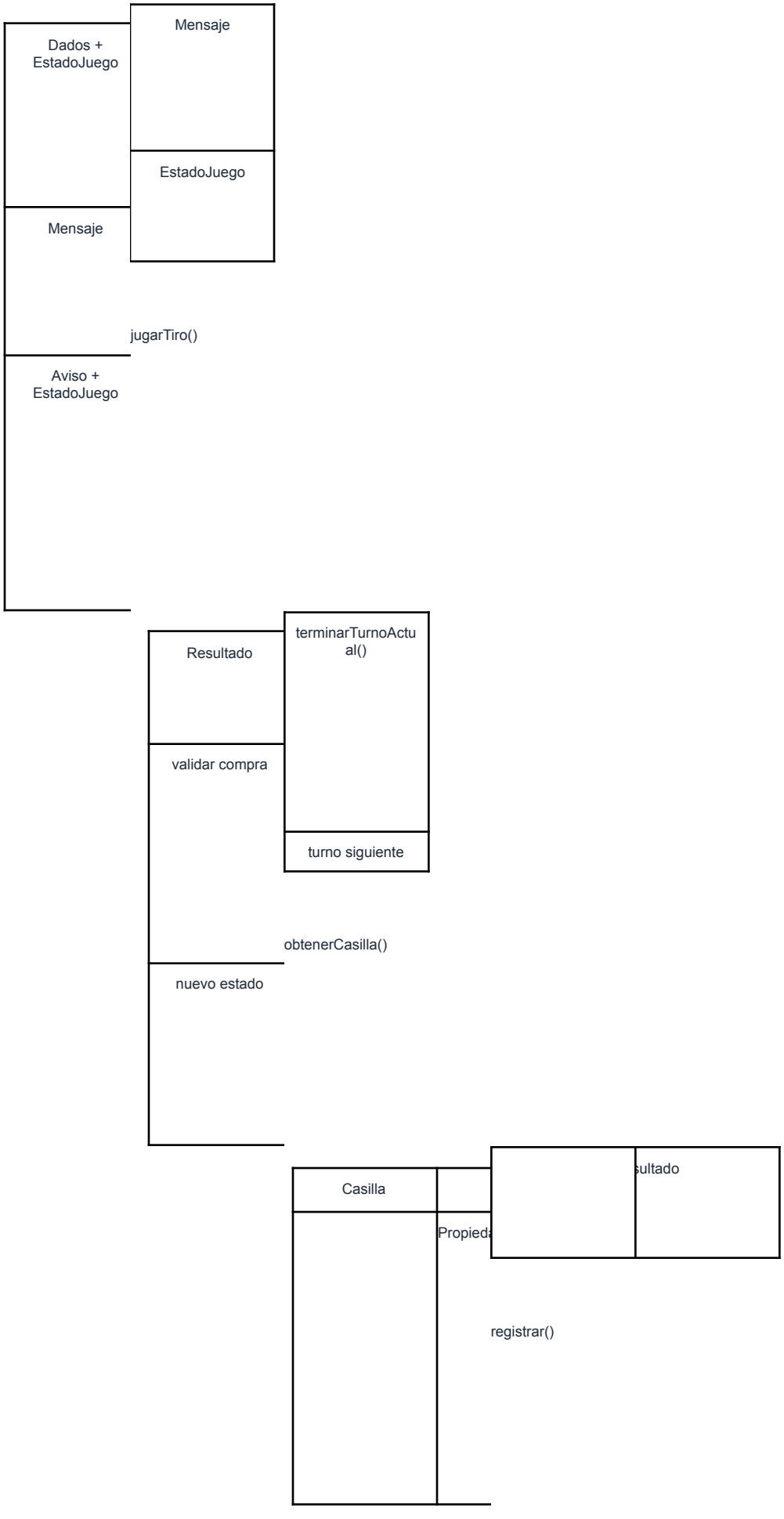
La Figura 5 resume una partida desde la solicitud hasta su registro.

Figura 5

Diagrama UML de secuencia: turno, casilla, compra y transacción

Solicitar tirar dados




Mensaje
Dados + EstadoJuego
EstadoJuego
Mensaje
jugarTiro()
Aviso + EstadoJuego
terminarTurnoActual()
Resultado
validar compra
turno siguiente
obtenerCasilla()
nuevo estado
sultado
Casilla
Propieda
registrar()

Nota. Secuencia representativa: las validaciones, el pago RFID y los eventos de otras casillas dependen de la acción y del estado del juego. Fuente: clases Cliente, Servidor, Juego, Tablero, Banco y RegistroTransacciones.

Errores y límites de la implementación actual

Si no hay respuesta al descubrimiento, se lanza IOException. Si vence la espera de Respuesta, Cliente informa que el servidor no respondió. Ante una desconexión, el hilo lector registra el error en consola; el manejador del servidor cierra el socket y retira la salida de la computadora en finally.

El bucle de aceptación del juego recibe únicamente dos conexiones y termina: no implementa reemplazo automático de una computadora desconectada. Cliente no contiene una rutina de reconexión automática ni un método explícito para cerrar su socket. No se observan TLS, autenticación de red ni un filtro de deserialización en estas clases.

La cola de Respuesta no utiliza identificadores de solicitud; presupone que las operaciones de gestión esperan sus respuestas en orden. Algunas llamadas de conexión y espera de respuestas se realizan directamente desde acciones de Interfaz, por lo que pueden bloquear temporalmente la interfaz aunque la lectura permanente esté en otro hilo.

Referencias

Instituto Tecnológico de Costa Rica. (2026). *Proyecto 1: Monopoly Distribuido con Estructuras Lineales* [Enunciado del proyecto]. Escuela de Ingeniería en Computación, Algoritmos y

Estructuras de Datos 1.

Monopoly distribuido: Código fuente de la conexión cliente-servidor. (s. f.). [Código fuente Java no publicado].

Apéndice

Inventario del código fuente consultado

Todas las rutas siguientes son relativas al proyecto; documentan el código revisado, sin introducir cambios:

src/main/java/Interfaz.java — main, iniciarServidor, iniciarRegistroTarjetas, abrirTablero.

src/main/java/Cliente.java — conectar, iniciarHiloLector, setOyente, enviar, recibirRespuesta.

src/main/java/Servidor.java — iniciar, escribir, difundir y operaciones de partida.

src/main/java/ManejadorCliente.java y ManejadorCajero.java — lectura y delegación.

src/main/java/Descubrimiento.java — anuncio y búsqueda UDP.

src/main/java/protocolo/Mensaje.java y config/Configuracion.java — contratos y puertos.

src/main/java/cliente/VentanaTablero.java — recibirMensaje y callbacks de acciones.

Anexos

Enlaces a los entregables complementarios del proyecto:

Código fuente (repositorio de GitHub):

https://github.com/jsandia/monopoly-datos1.git

Manual breve de ejecución (Google Docs):

https://docs.google.com/document/d/

1sCblzAmCTK_RgZcwuF-0xmHgOMW8hquS0iKx14TewDk/edit?usp=drive_link